PROGRAMAÇÃO FUNCIONAL

IO / AVALIZAÇÃO LAZY

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

ENTRADA/SAÍDA

Em disciplinas **tradicionais** de programação **imperativa** (como C, Java ou Python), a **primeira instrução** apresentada aos estudantes é a exibição de uma **mensagem na tela**: printf("Hello, World!"); ou print("Hello, World!").

ENTRADA/SAÍDA

No entanto, ao longo de **semanas** de estudo em **Haskell**, construímos **funções complexas**, **manipulação** de **tipos** e **estruturas recursivas** sem implementar um único comando de **exibição** na **tela**.

Por que o "Hello World" é diferente no paradigma **funcional** puro?

ENTRADA/SAÍDA

Natureza Matemática da Linguagem: Haskell fundamenta-se estritamente no **Cálculo Lambda** tipado. Um programa **não é** uma **sequência de instruções** para o processador executar, mas sim uma **expressão matemática** a ser **simplificada** (reduzida).

ENTRADA/SAÍDA

Ausência de Instruções Sequenciais: **Funções puras não** "fazem" coisas; elas **calculam** e **retornam** valores com **base** exclusiva em seus **argumentos.**

ENTRADA/SAÍDA

O Problema da Entrada/Saída (E/S): **Escrever** em um **terminal** ou **ler** um **arquivo** **altera** o **estado** do mundo exterior ao programa. Essa **alteração** é **denominada** **efeito colateral** (side effect).

FUNÇÕES PURAS E TRANSPARÊNCIA REFERENCIAL

Antes de introduzir a **sintaxe** de **E/S**, é necessário entender a **garantia matemática** que torna **programas** em Haskell **previsíveis** e **seguros** para **otimização** e **verificação** formal.

FUNÇÕES PURAS E TRANSPARÊNCIA REFERENCIAL

Transparência Referencial:
Uma **expressão** é dita **referencialmente transparente** se puder ser **substituída** pelo seu **valor resultante** sem alterar o **comportamento** ou **resultado** do programa.

Se **f(x) = y**, toda ocorrência de **f(x)** é intercambiável por **y**.

FUNÇÕES PURAS E TRANSPARÊNCIA REFERENCIAL

Propriedades de uma Função Pura:

1. **Determinismo**: Dados os **mesmos argumentos** de entrada, produz sempre o **mesmo resultado**.
2. **Isolamento**: **Não lê** e não **altera** nenhum estado **externo** (variáveis globais, arquivos, relógio do sistema, registradores de hardware).

O PARADOXO DO EFEITO COLATERAL E O TIPO

Se um **programa** estritamente **puro não** pode causar **efeitos colaterais**, como ele **interage** com o **usuário**, lê **discos** ou acessa **redes**? Um software **completamente isolado** do mundo **exterior** seria **inútil** na **prática**.

O PARADOXO DO EFEITO COLATERAL E O TIPO

A Solução de Haskell:
Em vez de permitir que **funções puras** executem **efeitos colaterais diretamente**, o Haskell **encapsula** ações do mundo real dentro de um **tipo especial** denominado **IO**.

O PARADOXO DO EFEITO COLATERAL E O TIPO

Semântica do Tipo IO a:
Representa uma "**receita**" ou "descrição de uma **ação**" que, quando **executada** pelo ambiente de execução (runtime) do **GHC**, realiza **operações externas** e produz um valor do tipo **a**.

O PARADOXO DO EFEITO COLATERAL E O TIPO

Separação de Papéis:

- **Código Puro**: Define a **lógica** de negócios, **cálculos** e **transformações** de dados.
- **Ações IO**: Agem como uma **camada** externa **interativa** que **coleta dados** do ambiente e os **entrega** ao núcleo puro.

PROGRAMAS INTERATIVOS E O PONTO DE ENTRADA

O Ponto de Entrada main:
Todo programa executável em **Haskell** deve conter uma **função** chamada **main** com o **tipo** estrito **IO** ().

PROGRAMAS INTERATIVOS E O PONTO DE ENTRADA

Nota(Construtor de Tipo Unitário ()):
O **tipo ()** (lido como unit) possui **apenas** um **valor**, (), e indica que a ação de **E/S** é **executada apenas** pelo seu **efeito colateral**, sem **retornar** um **valor computacional** útil.

PROGRAMAS INTERATIVOS E O PONTO DE ENTRADA

```
putStrLn :: String -> IO ()
getLine  :: IO String
```

PROGRAMAS INTERATIVOS E O PONTO DE ENTRADA

```
-- O ponto de entrada principal do programa
main :: IO ()
main = do
    putStrLn "Qual e o seu nome?"
    nome <- getLine
    putStrLn ("Ola, " ++ nome ++ "! Bem-vindo
a Programacao Funcional.")
```

AVALIAÇÃO SOB DEMANDA (LAZY EVALUATION)

Em **linguagens imperativas** e em muitas **funcionais** (como OCaml e Lisp), vigora a **Avaliação Estrita** (Eager/Strict Evaluation): **todos** os **argumentos** de uma **função** são **completamente computados** antes de a **função** ser **chamada**. **Haskell** adota a abordagem **oposta**: a **Avaliação Sob Demanda** (Lazy Evaluation).

AVALIAÇÃO SOB DEMANDA (LAZY EVALUATION)

Uma **expressão** só é **avaliada** no **momento exato** em que seu **resultado** se torna **estritamente necessário** para o **prosseguimento** da execução do programa (por exemplo, ao realizar pattern matching ou exibir um valor via E/S).

AVALIAÇÃO SOB DEMANDA (LAZY EVALUATION)

Atraso na Avaliação (Call-by-Name):
Adia o **cálculo** de **expressões** passadas a **parâmetros**.

Compartilhamento de Memória (Graph Reduction):
Garante que uma **expressão avaliada** uma vez não seja **reavaliada** se for utilizada **múltiplos locais**.

AVALIAÇÃO SOB DEMANDA (LAZY EVALUATION)

Conceito de Thunk:
Um **ponteiro** na **memória** contendo a **expressão não avaliada** e o **ambiente** de **variáveis necessárias** para **computá-la** posteriormente.

VANTAGENS PRÁTICAS E ESTRUTURAS DE DADOS INFINITAS

Qual a utilidade real de adiar cálculos?
A **avaliação sob demanda** permite **separar** o **algoritmo gerador** de dados da **quantidade** de **dados** consumidos, **viabilizando** o **uso seguro** de estruturas **potencialmente infinitas**.

VANTAGENS PRÁTICAS E ESTRUTURAS DE DADOS INFINITAS

Modularidade Aumentada:
É possível definir **listas infinitas** sem **esgotar** a **memória** do sistema.

Curto-Circuito Natural:
Funções de **filtragem** e **busca interrompem** o **processamento** imediatamente ao **encontrar** a **condição** necessária.

FOLDR VS FOLDL NA AVALIAÇÃO PREGUIÇOSA

O funcionamento dos dobradores de listas (foldr e foldl) já é conhecido. Contudo, sob o prisma da **Lazy Evaluation**, essas duas **funções** possuem **comportamentos assintóticos** e de **memória** completamente **divergentes**.

FOLDR VS FOLDL NA AVALIAÇÃO PREGUIÇOSA

```
foldr f z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

FOLDR VS FOLDL NA AVALIAÇÃO PREGUIÇOSA

Comportamento Preguiçoso:
A aplicação da função **f** ocorre na **camada externa**. Se a função **f não precisar avaliar** estritamente seu **segundo argumento** (a chamada recursiva foldr f z xs), a execução **pode terminar** antes de percorrer toda a lista.

FOLDR VS FOLDL NA AVALIAÇÃO PREGUIÇOSA

Compatibilidade com Listas Infinitas:
foldr funciona em **listas infinitas** caso f seja **não-estrita** no **segundo parâmetro**.

FOLDR VS FOLDL NA AVALIAÇÃO PREGUIÇOSA

```
foldl f z []     = z
foldl f z (x:xs) = foldl f (f z x) xs
```

FOLDR VS FOLDL NA AVALIAÇÃO PREGUIÇOSA

Cadeia de Thunks:
O **acumulador** (f z x) não é **computado imediatamente**. A **cada elemento** da lista, o **Haskell** apenas **envolve** o novo acumulador em um **novo Thunk**: f (f (f z x1) x2) x3.

FOLDR VS FOLDL NA AVALIAÇÃO PREGUIÇOSA

Consequência:
Para computar o **valor final** de uma lista longa, o programa **acumula** uma **pilha de expressões não avaliadas** na memória até que o caso base [] seja alcançado, causando estouro de memória (stack overflow) para listas grandes ou infinitas.

FOLDR VS FOLDL NA AVALIAÇÃO PREGUIÇOSA

> foldr eLogico True (False : repeat True)
False

> foldl eLogico True (False : repeat True)
-- Entra em loop infinito/estouro de memória

EXERCICIO

Escreva a função saudacaoInterativa :: IO () que lê o nome de um usuário via teclado utilizando getLine e exibe no terminal a mensagem "Ola, <NOME>! Bem-vindo ao GHCi." utilizando putStrLn.

EXERCICIO

Implemente a função dobrarNumero :: IO (). A função deve solicitar um número inteiro ao usuário, ler a entrada como String, converter para Int (com a função read) e exibir o resultado formatado no padrão "O dobro e: <VALOR>".

EXERCICIO

Uma fintech gera identificadores numéricos sequenciais pares (2, 4, 6, 8, ...) para transações.

1. Implemente a lista infinita gerarIDsPares :: [Integer].
2. Implemente a função selecionarLoteIDs :: Int -> [Integer] que recebe a quantidade n e retorna os n primeiros IDs utilizando take.

EXERCICIO

Dada uma sequência de checagens de segurança de rede representada por uma lista de booleanos, implemente a função verificarRedeSegura :: [Bool] -> Bool utilizando estritamente **foldr**. A função deve interromper o processamento imediatamente ao encontrar o primeiro valor **False** (propriedade de curto-circuito da avaliação sob demanda).

EXERCICIO

Considere as duas expressões abaixo aplicadas a uma lista infinita:

- Expressão A: foldr (&&) True (False : repeat True)
- Expressão B: foldl (&&) True (False : repeat True)

Explique **sucintamente** qual das duas chamadas **retorna** um **resultado** booleano e qual diverge (entra em loop infinito/estouro de pilha), **justificando** com base no **acúmulo de Thunks** e na estratégia de **Lazy Evaluation**.

DÚVIDAS???